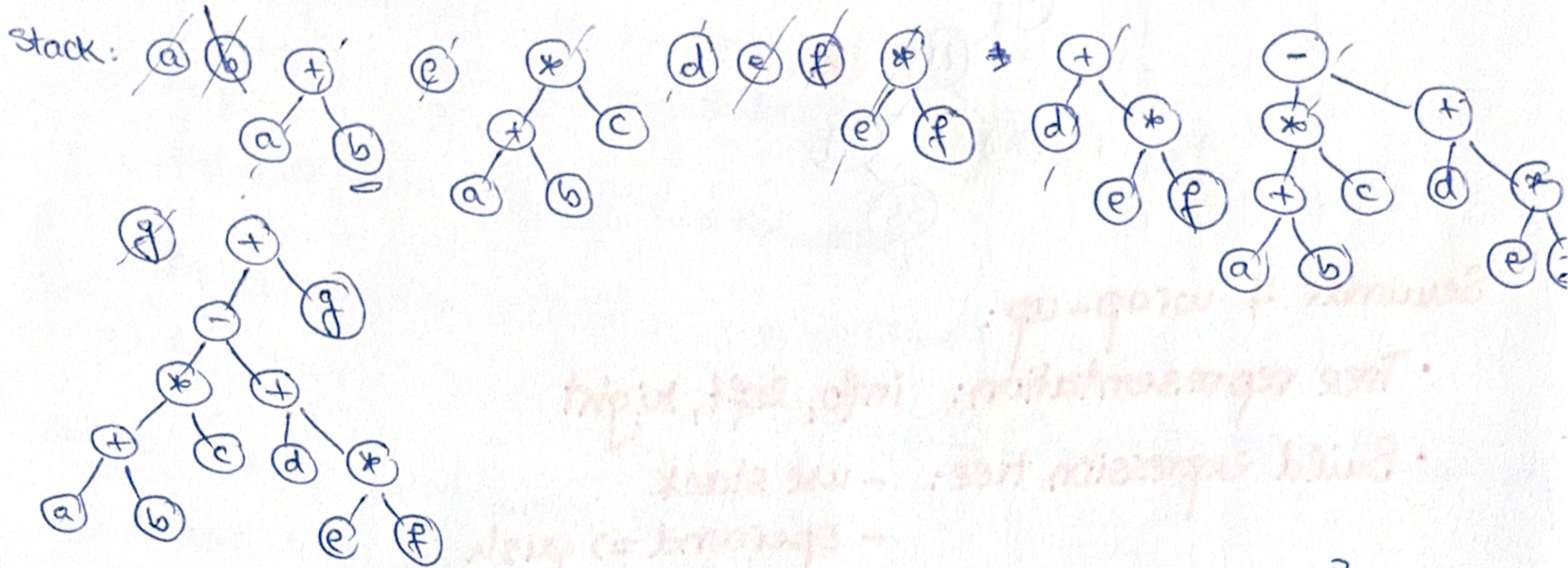


SCM7

1) BT for expression: $O(m)$

$(a+b)*c - (d+e*f)+g \Rightarrow$ postfix: $\overbrace{ab+c}^{} * \overbrace{def}^{} * + - g +$



2) BT for ancestors (left = maternal; right = paternal). All females (recursively):

subalg printFemales(bt) $\rightarrow$ follow left children.

init(q)

if bt.root $\neq$ Nil then

push(q, bt.root)

while not isEmpty(q) ex:

current $\leftarrow$ pop(q)

if [current].left $\neq$ Nil then

print([current].left.info)

push(q, [current].left)

if [current].right $\neq$ Nil then

push(q, [current].right)

print([bt.root].info) if root is female.

$O(h)$

$\leftarrow$ DFS

All ancestors (of degree k) (decrement k)

subalg levelK(node, k):

if node $\neq$ Nil then

if k = 0 then

print([node].info)

else

levelK([node].left, k-1)

levelK([node].right, k-1)

subalg printLevelK(bt, k):

levelK(bt.root, k)

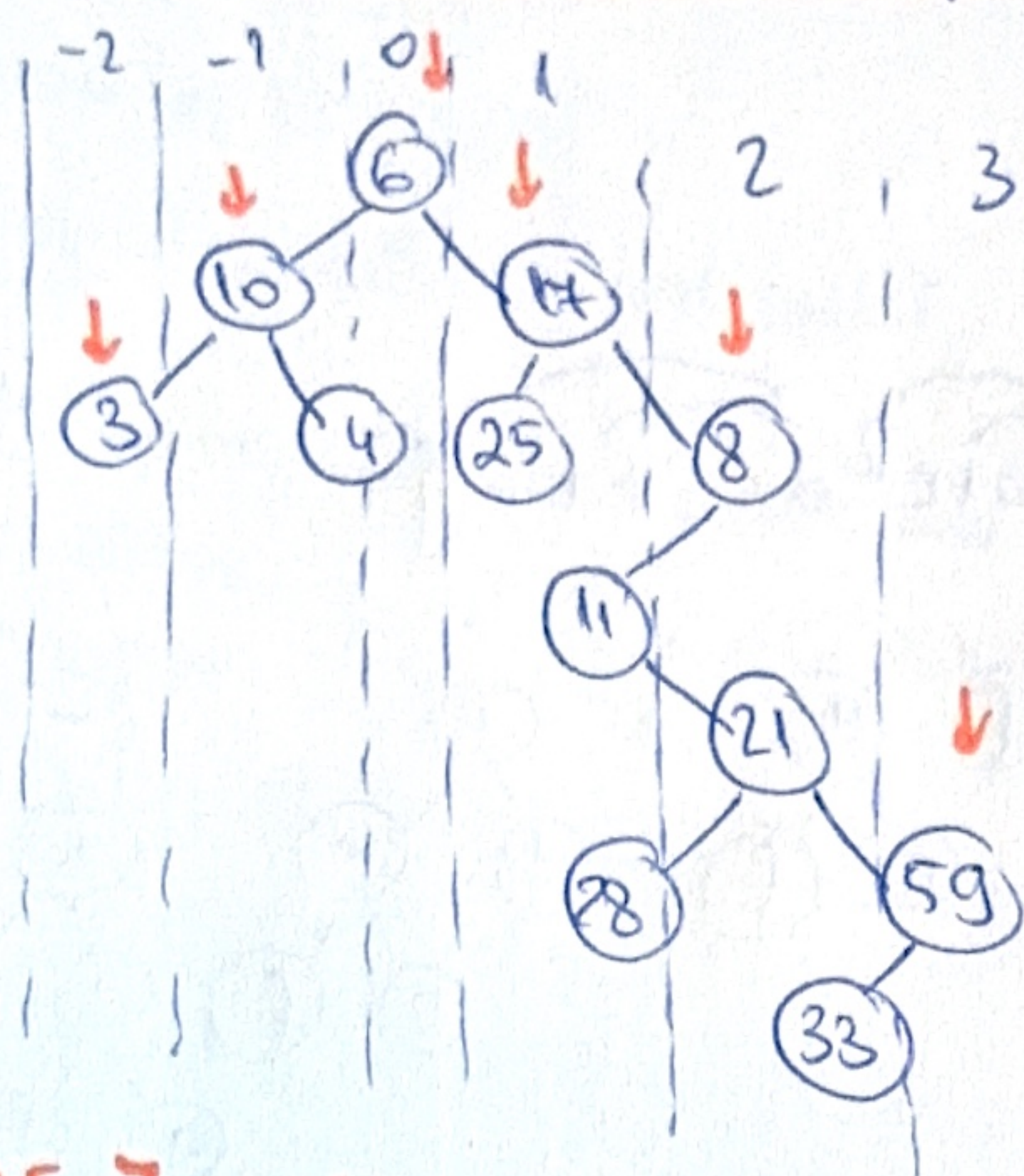
$O(m)$

$O(2^k) \rightarrow$ level k $\rightarrow$ at most 2^k nodes

$\Downarrow$

$O(\min(m, 2^k))$

3) top view = nodes visible from top of the tree



$\Rightarrow 3, 10, 6, 17, 8, 59$

$\Rightarrow$ queue of (node, col)

$\Rightarrow$ map : column $\rightarrow$ first node found

Seminar 7 wrap-up:

- Tree representation: info, left, right
- Build expression tree:
 - use stack
 - operand $\Rightarrow$ push
 - operator $\Rightarrow$ pop 2 elem; allocate, push
- Maximal time: node
 - node.left $\Rightarrow O(h)$
 - node.left.left...
- Print level K :
 - $K = 0 \Rightarrow$ print
 - else recurse with $K-1 \Rightarrow O(\min(m, 2^K))$
- Height of a tree: $1 + \max(\text{left}, \text{right}) \Rightarrow O(m)$
- Top view:
 - BFS + (node, col) $\Rightarrow O(m)$
 - first node in each col.

SEK6. 1) Iterator for a sorted map or HT, collision res. sep. chaining.

- hash by KEY only
- Keep each collision list sorted.

Method 1: merge all chains into one sorted list. $\Theta(m^2)$

m elems. $\left. \begin{array}{l} \} \\ m \text{ pos.} \end{array} \right\} \Rightarrow \frac{m}{m} = \alpha = \text{load factor} = \text{avg. length of a list}$

list 1 + list 2 $\Rightarrow$ list₁₂ $\Rightarrow$ (2α complexity)

!

(+) list 1 + ... + list m $\Rightarrow$ list_{1,2,...,m} $\Rightarrow$ ($m\alpha$)

$$\Rightarrow 2\alpha + 3\alpha + \dots = \alpha(2+3+\dots) \approx \alpha \cdot \frac{m(m+1)}{2} = \frac{m}{m} \cdot \frac{m(m+1)}{2} \in \Theta(m \cdot m)$$

Method 2: merge using heap: $\Theta(m \log m)$

→ min heap of nodes from the first node of every list.

2) BD Map

I. 2 hash tables:

key-table

value-table

23 $\rightarrow$ ball

ball $\rightarrow$ 23

II. STORE PAIR ONCE.

Node: